

Technische Hochschule Würzburg-Schweinfurt
Fakultät Kunststofftechnik und Vermessung

Anlage zur Projektarbeit

Dokumentation der KI-Nutzung

Scan-to-BIM-Pipeline – KI-gestützte 3D-Rekonstruktion eines linearen
Objekts

Vorgelegt von:	[Martin Rapps]
Matrikelnummer:	[6323014]
Studiengang:	[B. Eng. Geovisualisierung]
Erstprüfer/in:	[Dr. Markus Müller]
Zweitprüfer/in:	[Andreas Rupp]
Abgabetermin:	[08.09.2026]

Würzburg, den 27. August 2026

Hinweis zur Dokumentationsform

Auf Grund des Umfangs und der agentischen Arbeitsweise (selbstständige Änderungen über mehrere Dateien, iterative Überarbeitungen) wird die KI-Nutzung nach Bereichen beschrieben und jeweils durch beispielhafte Prompts veranschaulicht. Eine vollständige Auflistung aller Einzel-Prompts ist bei diesem Arbeitsmodus nicht sinnvoll und wurde so mit dem Prüfer abgestimmt.

Die Verantwortung für fachliche Richtigkeit, Eigenständigkeit und kritische Prüfung aller Inhalte liegt beim Verfasser. KI-Ausgaben wurden nicht als wissenschaftliche Quellen zitiert.

Inhaltsverzeichnis

1	Projektangaben	1
2	Eingesetzte Werkzeuge	1
3	Code-Erstellung / Verbesserung / Refactoring	1
4	Allgemeine Überlegungen zum Vorgehen (Sparringspartner)	2
5	Schreiben und Überarbeiten von Texten in der Projektarbeit	2
6	Fehleranalyse und Auswertung	2
7	Grafiken und Auswertung	3
8	Eigenständigkeit	3

1 Projektangaben

Projekt: Scan-to-BIM-Pipeline – KI-gestützte 3D-Rekonstruktion eines linearen C
 Verfasser: [Martin Rapps] ([6323014])
 Zeitraum der KI-Nutzung: Juni – August 2026
 Abgabe: [08.09.2026]

2 Eingesetzte Werkzeuge

Werkzeug	Version / Anbieter	Einsatzbereich
opencode (CLI-Agent)	aktuelle Version, verschiedene LLMs je nach Aufgabe	Code-Erstellung, Refactoring, Pipeline-Debugging, Versuchsplanung, Text- und Grafikarbeit
VS Code	aktuelle Version, integrierte KI-Unterstützung	Code-Editierung, Inline-Vorschläge, LaTeX-Bearbeitung

Tabelle 1: Übersicht der eingesetzten KI-Werkzeuge.

Die Modellwahl innerhalb von **opencode** erfolgte je nach Aufgabenkomplexität und Kosten – für einfache Korrekturen schlanke Modelle, für konzeptionelle oder fachliche Fragen leistungsfähigere Modelle.

Beispielhafte Modelle: Hauptsächlich genutzt wurden GPT-5.6 Luna (max), Gemini 3.5 Flash (max), Gemini 3.7 Flash (max), Kimi K3 (max), GLM 5.3 Flash (max) und GPT-5.6 Sol (max).

Alle KI-Ausgaben wurden manuell geprüft, verstanden und verantwortet.

3 Code-Erstellung / Verbesserung / Refactoring

Worum es ging: Aufbau und Pflege der Docker-basierten Pipeline (SAM-3.1-Segmentierung, COLMAP, STS, SuGaR-Fork, Centerline-Extraktion), insbesondere die Anpassung der Maskenlogik (Dilatation/Erosion für genau ein Zielobjekt), Fehlerbehandlung in Matrixläufen und die maskenbewusste Erweiterung des SuGaR-Forks.

Beispielhafte Prompts:

- „Die SAM-Masken sollen als Hierarchie vorliegen: **default** unverändert, **middle** einmal mit einem 5×5 -Fenster erodiert, **small** zweifach erodiert, ohne zusätzliche Dilatation. Stelle zusammen mit der promptbasierten Auswahl sicher, dass automatisch genau ein Zielobjekt übrig bleibt. Zeige nur die geänderte Funktion und erkläre die Parameter.“
- „Der Matrixrunner führt von meiner TSV-Konfiguration nur die erste Variante aus und beendet sich danach. Finde die Ursache und schlage eine Korrektur vor, die alle geplanten Läufe nacheinander ausführt.“
- „Im SuGaR-Fork soll der Verlust nur innerhalb der Objektmaske gewertet werden. Implementiere einen maskierten Loss mit Normierung über die Maskenpixel und Schutz vor Division durch null.“

Prüfung: Jeder Vorschlag wurde im lokalen Docker-Setup ausgeführt und gegen Manifeste/Logs geprüft; unverständene Änderungen wurden nicht übernommen.

4 Allgemeine Überlegungen zum Vorgehen (Sparingspartner)

Worum es ging: Struktur der Arbeit, Versuchsplanung (Matrixdesign Kameramodell $\times$ FPS $\times$ Auflösung), Bewertung von Varianten (Route A vs. SuGaR-Coarse, Tiefenrouten), Interpretation von Metriken.

Beispielhafte Prompts:

- „Ich habe die Kameramodelle PINHOLE und OPENCV getestet, beide liefern ähnliche Metriken. Erkläre fachlich, welche Argumente für welchen Standard sprechen und wo die Grenzen dieser Aussage ohne unabhängige Kalibrierung liegen.“
- „Hilf mir, die Matrixläufe so zu planen, dass ich Kameramodell, FPS und Auflösung getrennt auswerten kann, ohne die Meshroute zu vermischen.“

Prüfung: Vorschläge wurden mit dem Betreuer und dem Zielrahmen abgeglichen; fachliche Entscheidungen (z. B. Produktionsstandard) blieben eigene Arbeit.

5 Schreiben und Überarbeiten von Texten in der Projektarbeit

Worum es ging: Gliederung, Formulierung, Kürzung und LaTeX-Feinschliff (Tabellen, Abbildungen, Literatur, Overfull-Boxen).

Beispielhafte Prompts:

- „Dieser Absatz über die Domänentrennung ist zu lang. Gib mir ein Beispiel, wie ich ihn straffen kann, ohne die Kernbegriffe COLMAP, Maskenwarp und ideale Domäne zu verlieren.“
- „Formuliere diesen Satz wissenschaftlicher und ohne Füllwörter: ‚Das flexiblere Modell ist nicht grundsätzlich genauer, weil ...‘ – gib zwei Alternativen.“
- „Die Tabelle `tab:bildablagen` ist zu breit. Formatiere die Tabelle neu, ohne Inhalt zu verlieren.“

Prüfung: Alle Textvorschläge wurden satzweise gelesen, fachlich geprüft und in eigenen Worten überarbeitet; Zitate und Zahlen wurden gegen Manifeste und Tabellen verifiziert.

6 Fehleranalyse und Auswertung

Worum es ging: Diagnose fehlgeschlagener Läufe (leere Masken, SUGAR-Importfehler, COLMAP-Registrierung), Einordnung von Screenshots und Zwischenständen.

Beispielhafter Prompt:

- „Hier ein Log-Auszug mit leeren Eval-Masken. Welche Prüfung fehlt, und wo würdest du ein Quality-Gate einbauen? Erkläre den Vorschlag, bevor du Code änderst.“

Prüfung: Ursachen wurden im Code und in den Logs nachvollzogen; Korrekturen wurden erst nach manuellem Test übernommen.

7 Grafiken und Auswertung

Worum es ging: Erstellung der Panels (z. B. `pa_panel_sugar_iteration`), Metrikplots (STS vs. SuGaR, Delta, Laufzeit) und deren konsistente Beschriftung.

Beispielhafte Prompts:

- „Das Panel hat 9200er-Labels aus den historischen Dateinamen, soll aber konsistent 9001 zeigen. Schreibe ein Skript, das die Kacheln neu zusammensetzt und nur 9001 ausgibt.“
- „Lies aus den archivierten `sts_masked.json`-Dateien die objektmaskierten Werte PSNR, SSIM und LPIPS mit einem neuen Skript aus und erzeuge daraus eine übersichtliche Grafik je Kameramodell.“
- „Prüfe alle Metrikgrafiken auf konsistente Farb- und Symbolkodierung und korrigiere sie über alle Grafiken hinweg: `OPENCV` immer orange, `SIMPLE_RADIAL` immer blau, `PINHOLE` immer grau; je Auflösung ein eigener Formtyp (720p, QHD, low). Die Kodierung muss in jeder Grafik derselben Logik folgen.“

Prüfung: Generierte Grafiken wurden visuell und gegen die Quelldaten (JSON/CSVs) geprüft.

8 Eigenständigkeit

Alle Inhalte wurden fachlich geprüft, vollständig verstanden und können selbstständig vertreten werden. KI-Ausgaben wurden nicht als wissenschaftliche Quellen zitiert. Die Verantwortung für fachliche Richtigkeit und Eigenständigkeit liegt beim Verfasser.

Würzburg, den _____
Unterschrift des/der Studierenden